

GitHub setup and course workflow

14.33 Research and Communication in Economics · Fall 2026

Git tracks changes to files on your computer. GitHub stores a remote copy of the repository and supports collaboration, issues, and links to evidence. These instructions explain two ways to work with Git: the command line and GitHub Desktop, a user interface application.

1. Create a repository from the course template

1. Create a free account at github.com, or sign in to your existing account.
2. Open the [14.33 project template](#).
3. Select **Use this template** → **Create a new repository**.
4. Select your personal GitHub account as the owner.
5. Name the repository `fa26-1433-YOUR-KERBEROS`. Replace `YOUR-KERBEROS` with your MIT Kerberos username.
6. Use the repository visibility specified in class, then select **Create repository**.
7. In your new repository, select **Settings** → **Collaborators**. Add the instructor and TA using the GitHub usernames provided in class. This access is required by the [course syllabus](#).

Your new repository contains the course directory structure and README files. It is separate from the original template. Do not commit confidential data, passwords, API keys, or other secrets.

2. Choose how to work with Git

You can use either method throughout the course. Both methods change the same files and Git history.

GitHub Desktop

Install [GitHub Desktop](#) and sign in to your GitHub account. GitHub Desktop provides buttons and menus for the most common Git actions.

Command line

The command line lets you run Git commands in a terminal.

Windows

Install [Git for Windows](#). Keep Git Credential Manager selected during installation.

macOS

If you use [Homebrew](#), run:

```
brew install git
brew install --cask git-credential-manager
```

Git Credential Manager opens a browser when GitHub authentication is required. Follow the browser prompts. See [GitHub's authentication instructions](#) if authentication fails.

Identify your command-line commits

Run these commands once. Replace the example values with your name and the email address associated with GitHub.

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

3. Clone your repository

Cloning downloads the repository to your computer and connects the local copy to GitHub.

GitHub Desktop

1. On your repository's GitHub page, select **Code** → **Open with GitHub Desktop**.
2. Select the local folder where you want to keep the project.
3. Select **Clone**.

Command line

On the GitHub repository page, select **Code** → **HTTPS**, then copy the URL. In a terminal, run:

```
git clone REPOSITORY-HTTPS-URL
cd fa26-1433-YOUR-KERBEROS
```

4. Use GitHub issues for the course

Use two types of issues:

- A **meeting agenda issue** summarizes the project before one scheduled meeting. Create a new agenda for each meeting.
- A **work issue** tracks one consequential decision, question, blocker, or incomplete task. Keep the issue open across meetings until you resolve it.

Create a meeting agenda issue

At least 24 hours before each one-on-one meeting:

1. Select **Issues** → **New issue** in your project repository.
2. Next to **Meeting preparation**, select **Get started**.
3. Add the meeting date to the issue title: `Meeting preparation – YYYY-MM-DD`.
4. Complete each section of the issue template. Aim for approximately 250 words, excluding links and tables.
5. Link each finding to supporting code, output, or documentation in the repository.
6. In **Decisions, questions, or blockers**, link each relevant work issue by typing its number, such as #23.
7. Select **Submit new issue**.

Report findings and decisions rather than time spent. Use the agenda as a concise guide for the meeting, not as the complete history of the project.

Create and link a work issue

Create a separate work issue when an item needs evidence, discussion, or follow-up beyond one meeting:

1. Select **Issues** → **New issue**.
2. Next to **Project decision or question**, select **Get started**.
3. Give the issue a specific title, such as `Choose the appropriate sample definition`.
4. Complete the template with the relevant evidence, alternatives, and your current recommendation.
5. Select **Submit new issue**.
6. Link the work issue from the next meeting agenda by typing its issue number.

You may test important alternatives on short-lived branches. Link each branch from the applicable work issue. After the meeting, update the agenda with agreed next steps. Record each decision and its rationale in the applicable work issue, then close that issue when the work is complete.

5. Save work and correct mistakes with Git

Git records snapshots called **commits**. A commit exists only in your local repository until you **push** it to GitHub. **Pull** downloads commits that are on GitHub but not on your computer.

GitHub Desktop workflow

1. Select **Fetch origin**. If GitHub Desktop shows **Pull origin**, select it.
2. Review the files and line-by-line changes in the **Changes** tab.
3. Clear the checkbox for any file that does not belong in the next commit.
4. Enter a short summary of the completed change.
5. Select **Commit to main**.
6. Select **Push origin**.

If you must reverse a committed change, open **History**, right-click the commit, and select **Revert Changes in Commit**. Push the new reversal commit.

Command-line workflow

```
git pull --ff-only
git status
git add path/to/file
git diff --staged
git commit -m "Describe the completed change"
git push
```

To remove a file from the next commit without deleting your changes, run:

```
git restore --staged path/to/file
```

To reverse a commit that is already on GitHub, preserve the shared history:

```
git revert COMMIT-ID
git push
```

Practices for both workflows

- Review each commit before you create it.
- Make small, coherent commits with summaries such as `Add baseline summary statistics`.
- Commit source code, documentation, and small outputs needed to understand the project.
- Do not commit large raw datasets unless the course staff approves them.
- Never commit confidential data, passwords, API keys, or other secrets.
- Keep `main` usable. Use a short-lived branch to test an important alternative.
- Ask the instructor or TA before you rewrite published history with `git reset --hard` or a force push.